

Problem 1: Weird Tree

Approach

I performed a single breadth-first traversal of the tree using a queue, tracking the current depth. For even layers, node IDs are appended directly to the output in level-order. For odd layers, node IDs are collected per layer. After the traversal completes, the odd layers are appended in reverse order (bottom-up), and each layer is reversed to achieve right-to-left ordering. This satisfies the traversal order using one pass over the tree structure.

What I tested

- Single-node tree.
- Empty tree.
- Degenerate line-shaped tree.
- Wide tree with many siblings.
- Unbalanced tree with mixed depth and branching.

Problem 2: Bits-to-Tree

Approach

I first validated the bit sequence by checking prefix constraints and ensuring the number of 1s equals the number of 0s. I then reconstructed the tree using a stack to simulate the DFS traversal: a 1 creates a new child and descends, while a 0 ascends to the parent. Nodes are labeled incrementally in preorder. The first-child/next-sibling representation is maintained by tracking the most recently added child for each node.

What I tested

- Empty bit vector producing a single root node.
- Invalid prefix cases where upward traversal occurs too early.
- Invalid encodings with mismatched numbers of up and down traversals.

Problem 3: Light Post (Cameras pt 2)

Approach

I did almost the same as cameras solution. For each node, I did three DP states: placing a light at the node, being covered by a child, or being covered by the parent. A post-order DFS computes minimum costs for each state. A second DFS reconstructs the chosen set of nodes based on the optimal states. This runs in $O(n)$.

What I tested (had AI generate these cases based on some of leetcode's public "camera" cases)

- Path-shaped tree where placing lights on internal nodes is cheaper than endpoints.
- Star-shaped trees favoring either the center or all leaves depending on costs.
- Cases where the optimal solution places a light at the root versus at the children

Problem 4: Smallest Subset Sum

Approach

DP where $dp[i][j]$ stores the minimum subset size needed to form sum j using elements from index i onward. This ensures $O(n * T)$ time. After filling the table, I reconstructed the solution from $L \rightarrow R$, only choosing an element if it preserves optimal subset size. This guarantees both minimum cardinality and lexicographically smallest order.

What I tested

- Single-element solutions where one value equals the target.
- Multiple size-1 solutions, ensuring the earliest index is chosen.
- Multiple minimum-size solutions of size greater than one, checking lexicographic order.
- Duplicate values, verifying earlier indices are preferred.
- Cases with no valid subset summing to the target.